Deliverable Sprint 1 – Aplicație pentru Închirieri și Service Auto

1. Tema aleasă

Sistem integrat care unifică operațiunile de **închirieri auto** cu managementul unui **service auto**. Aplicația permite gestionarea flotei de vehicule, crearea și urmărirea contractelor de închiriere, programarea și monitorizarea lucrărilor de service, precum și gestionarea clienților și a angajaților. Scopul este eliminarea redundanței între cele două activități și oferirea unei vizibilități complete asupra stării fiecărui vehicul.

2. Echipa

Pietrișl Paul-Valentin – Java Backend Developer

Responsabil pentru implementarea stratului de business logic în Spring Boot: entități JPA, servicii REST, repository-uri, autentificare și securitate (Spring Security).

Botezatu Elena-Christiana– Frontend Developer

Responsabilă pentru interfața utilizator: pagini HTML/CSS/JavaScript, consumul API-urilor REST prin Fetch API, design responsiv și UX pentru modulele de închirieri și service.

Voia Bianca-Elena – Database & QA Engineer

Responsabil pentru proiectarea schemei relaționale PostgreSQL, scrierea scripturilor SQL (DDL/DML), migrări Flyway, și scrierea testelor unitare/integrare (JUnit 5, Mockito).

3. Backlog inițial

Epics

- EP-01: Gestionare vehicule (flotă)
- EP-02: Gestionare închirieri
- EP-03: Gestionare service auto
- EP-04: Gestionare clienți
- EP-05: Raportare și statistici

User Stories (scrise în Gherkin)

Feature: Adăugare vehicul în flotă

```
Scenario: Administrator adaugă un vehicul nou
  Given că sunt autentificat ca administrator
  When completez numărul de înmatriculare, marca, modelul și anul
  Then vehiculul apare în lista flotei cu statusul "Disponibil"
```

Feature: Creare contract de închiriere

```
Scenario: Client rezervă un vehicul disponibil
  Given că există cel puțin un vehicul cu statusul "Disponibil"
```

```
  When clientul selectează vehiculul, data start și data retur
  Then se generează un contract de închiriere
  And vehiculul primește statusul "Închiriat"
```

Feature: Returnare vehicul

```
Scenario: Operator înregistrează returnarea unui vehicul
  Given că există un contract activ pentru vehiculul respectiv
  When operatorul marchează vehiculul ca returnat
  Then contractul se închide cu data reală de retur
  And se calculează costul final pe baza zilelor efective
  And vehiculul revine la statusul "Disponibil" sau "În service"
```

Feature: Programare lucrare de service

```
Scenario: Mecanic creează o fișă de service
  Given că vehiculul nu este activ într-un contract de închiriere
  When mecanicul introduce tipul lucrării, piesele și manopera estimată
  Then se creează o fișă de service și vehiculul ia statusul "În
service"
```

Feature: Finalizare lucrare de service

```
Scenario: Mecanic finalizează o lucrare
  Given că există o fișă de service activă pentru vehicul
  When mecanicul marchează lucrarea ca finalizată și introduce costul
real
  Then fișa se închide cu data finalizării
  And vehiculul revine la statusul "Disponibil"
```

Feature: Vizualizare disponibilitate flotă

```
Scenario: Administrator verifică starea flotei
  Given că sunt autentificat
  When accesez pagina de dashboard
  Then văd lista vehiculelor cu statusul curent
  And pot filtra după status: Disponibil / Închiriat / În service
```

Feature: Gestionare clienți

```
  Scenario: Administrator înregistrează un client nou
    Given că sunt autentificat ca administrator
    When introduc datele necesare ale clientului
    Then clientul este salvat în sistem
    And apare în lista clienților
```

Feature: Raportare și statistici

```
  Scenario: Administrator vizualizează situația flotei
    Given că sunt autentificat ca administrator
    When accesez secțiunea de rapoarte
    Then văd numărul vehiculelor disponibile,
    închiriate și aflate în service
```

4. ADR 001 – Alegerea stack-ului tehnologic

Context: Echipa trebuie să aleagă un stack tehnologic potrivit pentru un sistem integrat cu backend, frontend și bază de date relațională.

Opțiuni analizate:

- Java + Spring Boot + PostgreSQL + HTML/CSS/JS (Vanilla)
- .NET Core + SQL Server + Angular
- Node.js + MongoDB + React

Decizie: Se alege **Java 21 + Spring Boot 3 + PostgreSQL + HTML/CSS/JavaScript** deoarece:

- Java și Spring Boot sunt cunoscute de Paul și reprezintă standardul industry pentru aplicații enterprise.
- PostgreSQL este open-source, robust și potrivit pentru date relaționale (vehicule, contracte, fișe service).
- HTML/CSS/JS Vanilla reduce complexitatea toolchain-ului (fără build step), ușor de gestionat de Bianca.
- Spring Boot oferă integrare nativă JPA/Hibernate, Spring Security și generare facilă de REST API.
- Maven asigură dependency management standardizat, conform cursului (C1/C2).
- IntelliJ IDEA Ultimate (disponibil prin licența UAIC) oferă suport complet pentru toate tehnologiile alese.

Consecințe: Curbă de învățare redusă, ecosistem matur cu documentație abundentă, posibilitate de extindere cu Spring Security, Flyway și testare automată cu JUnit 5 + Mockito.

Stack detaliat:

Layer	Tehnologie
Backend	Java 21, Spring Boot 3, Spring MVC, Spring Data JPA, Spring Security
Frontend	HTML5, CSS3, JavaScript (ES6+), Fetch API
Bază de date	PostgreSQL 16, Flyway (migrări), SQL
Build	Maven 3, pom.xml (conform C1/C2)
IDE	IntelliJ IDEA Ultimate
Versionare	Git + GitHub
Testare	JUnit 5, Mockito

5. C4 Context Diagram

Actorii principali ai sistemului:

- **Administrator** – gestionează flota, contractele, fișele de service și utilizatorii.
- **Client** – rezervă vehicule, vizualizează contractele proprii și istoricul.

- **Mecanic** – creează și finalizează fișe de service, actualizează statusul vehiculelor.

Sistemul principal: Aplicație Închirieri și Service Auto

- Modul Flotă: gestionare vehicule, status, istoricul complet pe vehicul.
- Modul Închirieri: contracte, calcul tarife, disponibilitate în timp real.
- Modul Service: fișe de lucru, piese, manoperă, istoricul intervențiilor.
- Modul Clienți: date personale, permis de conducere, istoricul închirierilor.
- Backend REST API: Spring Boot expune endpoint-uri JSON consumate de frontend.
- Baza de date: PostgreSQL stochează toate entitățile relaționale.

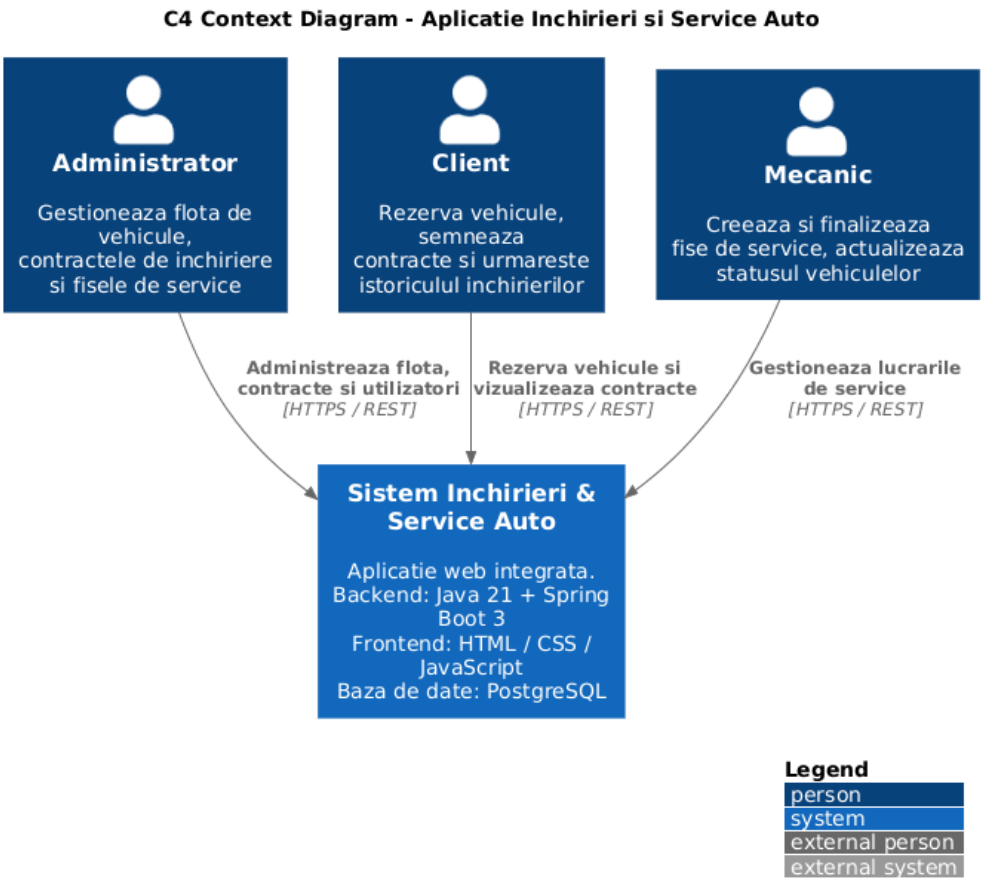
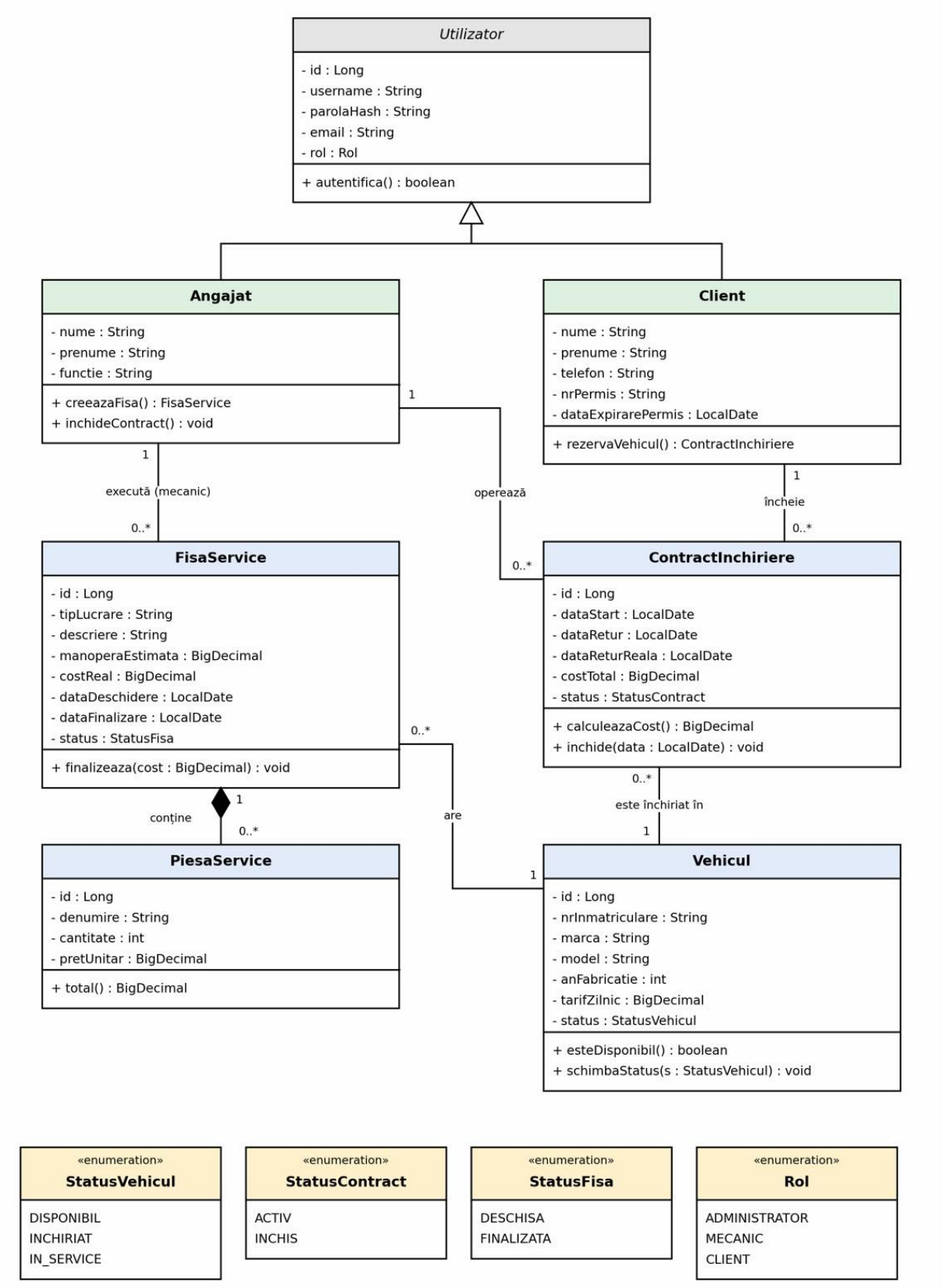


Diagrama a fost generată cu PlantUML C4 Model (https://www.plantuml.com/plantuml/uml/).


Utilizator
- id : Long
- username : String
- parolaHash : String
- email : String
- rol : Rol
+ autentifica() : boolean
Angajat
- nume : String
- prenume : String
- functie : String
+ creeazaFisa() : FisaService
+ inchideContract() : void
Client
- nume : String
- prenume : String
- telefon : String
- nrPermis : String
- dataExpirarePermis : LocalDate
+ rezervaVehicul() : ContractInchiriere
1
execută (mecanic)
0..*
operează
1
0..*
încheie
1
0..*
FisaService
- id : Long
- tipLucrare : String
- descriere : String
- manoperaEstimata : BigDecimal
- costReal : BigDecimal
- dataDeschidere : LocalDate
- dataFinalizare : LocalDate
- status : StatusFisa
+ finalizeaza(cost : BigDecimal) : void
ContractInchiriere
- id : Long
- dataStart : LocalDate
- dataRetur : LocalDate
- dataReturReala : LocalDate
- costTotal : BigDecimal
- status : StatusContract
+ calculeazaCost() : BigDecimal
+ inchide(data : LocalDate) : void
conține
1
0..*
0..*
are
1
0..*
este închiriat în
1
PiesaService
- id : Long
- denumire : String
- cantitate : int
- pretUnitar : BigDecimal
+ total() : BigDecimal
Vehicul
- id : Long
- nrInmatriculare : String
- marca : String
- model : String
- anFabricatie : int
- tarifZilnic : BigDecimal
- status : StatusVehicul
+ esteDisponibil() : boolean
+ schimbaStatus(s : StatusVehicul) : void
«enumeration»
StatusVehicul
DISPONIBIL
INCHIRIAT
IN_SERVICE
«enumeration»
StatusContract
ACTIV
INCHIS
«enumeration»
StatusFisa
DESCHISA
FINALIZATA
«enumeration»
Rol
ADMINISTRATOR
MECANIC
CLIENT